shared by developers or testers; it should be reserved for the exclusive use of your automated acceptance criteria.

Many teams today try to automate this provisioning process using tools like Puppet and Chef to create a clean test environment from scratch on a freshly created virtual machine before each test run.

Any test that uses a database will typically need the database to be in a known, predictable state at the start of the test. When it comes to setting up your test database, there are several possible approaches; the one that works best for you will largely depend on the nature of your project and technical environment. In the following sections, we'll discuss a few possible strategies.

7.2.1 Initializing the database before each test

The most reliable way to set up your test database is to automatically reinitialize the database schema before each test, possibly populating it with a sensible predefined set of reference data. This way, no test can inadvertently break another test by adding unexpected data into the database. It also ensures that each scenario is independent of the others and doesn't rely on another scenario to have been executed beforehand.

7.2.2 Initializing the database at the start of the test suite

Unfortunately, in practice your database or technology stack may make initializing the database schema before each test too slow to be a viable option. I've seen several organizations use this approach very successfully, even with large relational databases, but fast feedback is important, and you may have to make trade-offs in this area.

The next-best way to prepare test data is to automatically reinitialize the database schema every time you run the test suite. This is faster than reinitializing the database before each scenario, but it means that each scenario is responsible for deleting any test data that it creates, which isn't without risk. This approach can also introduce subtle dependencies between the scenarios, which can lead to hard-to-reproduce errors.

7.2.3 Using initialization hooks

No matter which option you chose, most BDD tools provide "hooks" that allow you to perform actions before and after each scenario and at other strategic points in the test suite lifecycle.

As you've seen, BDD tools like JBehave express requirements in the form of scenarios. When the tests are executed, a scenario will generally correspond to a single automated test, although for table-driven scenarios a test will be executed for each row in the table. The scenarios are placed in feature files or story files. I'll refer to all of these story and feature files as the test suite.

Different tools let you intervene at slightly different stages of test execution, but the main steps in test execution are similar for all of the BDD tools we'll discuss, and they lead to the intervention points illustrated in figure 7.5.

Table 7.1 illustrates some of the methods available in the various tools for intervening at each of these points.